Claude

I'm doing a Pygame Frogger lab. The game has one deliberate bug: vehicle collision detection compares grid columns instead of checking whether the frog and a vehicle actually overlap, so the frog sometimes passes through vehicles, especially in faster lanes. Task 1: fix collision detection so it uses the real positions and sizes of the frog and vehicles. Only fix collision detection. Don't add lives, respawn, score or a timer yet. Explain what was wrong, then show only the changed code.

game/collisions.py contains

```
"""
collisions: frog-vs-vehicle collision detection.
"""

CELL_SIZE = 50

def check_collision(frog, vehicles):
    """
    Returns True if the frog is currently hit by any vehicle.
    """
    for v in vehicles:
        vehicle_col = int(v.x // CELL_SIZE)
        if vehicle_col == frog.col and v.row == frog.row:
            return True
    return False
```

`game/frog.py`contains

```
"""
Frog: the player-controlled character. Moves in discrete grid hops,
one cell per key press - not continuous pixel movement.
"""

import pygame
```

```
class Frog:
    def __init__(self, col, row, start_col, start_row, cols, start_row_limit):
        self.col = col
        self.row = row
        self.start_col = start_col
        self.start_row = start_row
        self.cols = cols
        self.start_row_limit = start_row_limit  # frog can't move below its own starting row

    def move(self, dcol, drow):
        new_col = self.col + dcol
        new_row = self.row + drow
        if 0 <= new_col < self.cols:
            self.col = new_col
        if 0 <= new_row <= self.start_row_limit:
            self.row = new_row

    def reset(self):
        self.col = self.start_col
        self.row = self.start_row

    def get_rect(self, cell_size, pad=4):
        return pygame.Rect(
            self.col * cell_size + pad,
            self.row * cell_size + pad,
            cell_size - 2 * pad,
            cell_size - 2 * pad,
        )
```

`game/vehicle.py` contains

```
"""
Vehicle: a car/truck on a road lane. Moves continuously in pixel space
(not grid-snapped like the frog), wraps around when it exits the screen.
"""

import pygame
```

```
class Vehicle:
    def __init__(self, x, row, width, height, speed):
        self.x = float(x)
        self.row = row
        self.width = width
        self.height = height
        self.speed = speed    # px/frame, negative = moving left

    def update(self, road_width_px):
        self.x += self.speed
        if self.speed > 0 and self.x > road_width_px:
            self.x = -self.width
        elif self.speed < 0 and self.x < -self.width:
            self.x = road_width_px

    def get_rect(self, cell_size, pad=4):
        return pygame.Rect(
            int(self.x),
            self.row * cell_size + pad,
            self.width,
            cell_size - 2 * pad,
        )
```

`game/game_engine.py` contains

```
"""
GameEngine: owns the frog and all vehicles, and runs one frame's worth
of game logic.

Starter version: the frog can move, hop across the road, and reach the
goal - but there's no lives system, no score, and no timer. Collision
detection also has a known bug (see game/collisions.py) that Task 1
asks you to fix.
"""

import random

import pygame
```

```
from game.frog import Frog
from game.vehicle import Vehicle
from game.collisions import check_collision
from game.renderer import (
    GRID_COLS, GRID_ROWS, GOAL_ROW, ROAD_ROWS, START_ROW, CELL_SIZE, WIDTH,
HEIGHT,
)

LANE_SPEEDS = [1.5, -2, 2, -2.5, 1.5, -2]   # one entry per road row, alternating direction

class GameEngine:
    def __init__(self):
        self._build_entities()

    def _build_entities(self):
        start_col = GRID_COLS // 2
        self.frog = Frog(
            col=start_col, row=START_ROW,
            start_col=start_col, start_row=START_ROW,
            cols=GRID_COLS, start_row_limit=START_ROW,
        )
        frog_x_range = (start_col * CELL_SIZE, start_col * CELL_SIZE + CELL_SIZE)

        self.vehicles = []
        for i, row in enumerate(ROAD_ROWS):
            speed = LANE_SPEEDS[i % len(LANE_SPEEDS)]
            vehicle_width = 40 if i % 2 == 0 else 70    # mix of cars and wider trucks
            spacing = 300
            count = 2

            # Try a few random phases and keep the first one that doesn't
            # already overlap the frog's starting column - guarantees a
            # safe first lane instead of leaving it to chance.
            for _attempt in range(20):
                phase = random.randint(0, spacing - 1)
                positions = []
                safe = True
```

```
            for n in range(count):
                offset = phase + n * spacing
                x = offset if speed > 0 else WIDTH - offset - vehicle_width
                positions.append(x)
                if not (x + vehicle_width <= frog_x_range[0] or x >= frog_x_range[1]):
                    safe = False
            if safe:
                break

        for x in positions:
            self.vehicles.append(Vehicle(x=x, row=row, width=vehicle_width,
                                         height=CELL_SIZE - 8, speed=speed))

def handle_keydown(self, key):
    if key == pygame.K_UP:
        self.frog.move(0, -1)
    elif key == pygame.K_DOWN:
        self.frog.move(0, 1)
    elif key == pygame.K_LEFT:
        self.frog.move(-1, 0)
    elif key == pygame.K_RIGHT:
        self.frog.move(1, 0)
    elif key == pygame.K_r:
        self._build_entities()

def update(self):
    for v in self.vehicles:
        v.update(road_width_px=WIDTH)

    if check_collision(self.frog, self.vehicles):
        self.frog.reset()

    if self.frog.row == GOAL_ROW:
        self.frog.reset()

def draw(self, surface, font):
    from game import renderer
```

renderer.draw_scene(surface, self.frog, self.vehicles)
renderer.draw_text(surface, font, "Arrow keys to move. R to restart.", (10, HEIGHT - 24))

Three files are unchanged (`collisions.py`, `frog.py`, `vehicle.py`), and `renderer.py` needs nothing either. Everything is in `game/game_engine.py`. I didn't have your `main.py`, so the engine measures elapsed time itself with `time.perf_counter()` instead of needing a `dt` passed in. That means `main.py` doesn't change.

1. `game/game_engine.py`: add an import

Put it next to `import random`:

python

```python
import time
```

2. `game/game_engine.py`: add a constant

Below `SCORE_PER_CROSSING`:

python

```python
TIME_LIMIT = 30.0          # seconds per attempt
MAX_DT = 0.1               # cap on a single frame's elapsed time (guards against stalls)
```

3. `game/game_engine.py`: add state in `_build_entities`

Add these lines with the other state lines at the top of the method. `R` already calls this method, so it resets the timer too.

python

```python
        self.time_left = TIME_LIMIT
        self._last_tick = time.perf_counter()
```

```
        self.hit_reason = None      # "vehicle" or "time", for
the on-screen indication
```

4. `game/game_engine.py`: replace `update`

python

Python

```
   def update(self):
        # Real elapsed time since the previous update (not
frame counting).
        now = time.perf_counter()
        dt = min(now - self._last_tick, MAX_DT)
        self._last_tick = now

        # Vehicles keep moving the whole time, even during the
hit effect.
        for v in self.vehicles:
            v.update(road_width_px=WIDTH)

        # After a win or game over: no collisions, no life
changes, timer frozen.
        if self.game_over or self.won:
            return

        # Hit effect running: timer frozen, no collision
checks, respawn when it ends.
        if self.hit_timer > 0:
            self.hit_timer -= 1
            if self.hit_timer == 0:
                self.frog.reset()
                self.hit_reason = None
                self._last_tick = time.perf_counter()   # new
attempt starts now
            return

        if check_collision(self.frog, self.vehicles):
```

```
            self._lose_life("vehicle")
            return

        # Goal reached: register the crossing, add score, end
the game.
        # Checked before the timer so reaching the goal always
beats the clock.
        if self.frog.row == GOAL_ROW:
            self.score += SCORE_PER_CROSSING
            self.won = True


           return

        # Countdown for the current attempt.
        self.time_left -= dt
        if self.time_left <= 0:
            self.time_left = 0.0
            self._lose_life("time")

    def _lose_life(self, reason):
        self.lives -= 1
        self.hit_reason = reason
        if self.lives <= 0:
            self.game_over = True
        else:
            self.hit_timer = HIT_DURATION_FRAMES
            self.time_left = TIME_LIMIT      # each attempt
gets its own 30 seconds
```

`_lose_life` is a new method, so put it inside the `GameEngine` class right after `update`, at the same indentation. The collision branch now calls it, so the lives behaviour from Task 2 is the same as before.

5. `game/game_engine.py`: edit `draw`

Add the timer line next to the Lives and Score lines:

python

Python

```
        renderer.draw_text(surface, font, f"Time:
{math.ceil(self.time_left)}", (WIDTH // 2 - 40, 10))
```

This needs `import math` at the top of the file too, next to `import time`.

Then add the “Time’s Up!” indication in the banner section at the bottom of `draw`, so the whole block reads:

python

Python

```
        # Banners are drawn last so nothing covers them.
         if self.game_over:
             renderer.draw_banner(surface, "Game Over")
         elif self.won:
             renderer.draw_banner(surface, "You Won!")
         elif self.hit_timer > 0 and self.hit_reason == "time":
             renderer.draw_banner(surface, "Time's Up!",
size=48)
```

How it behaves

- Start: the timer runs only while the frog can move. It starts at 30 on launch and after each respawn.
- Timeout: one life is lost, the frog turns red (the Task 2 effect) with a “Time’s Up!” banner, then it respawns and the timer resets to 30. On the last life it goes straight to “Game Over”.
- Vehicle hit: the timer resets to 30 and stays frozen while the hit effect plays.
- Win: the goal check runs before the timer, so reaching the goal always wins, even on the same frame the clock would hit zero. The timer then freezes.
- Game over and win: the timer is frozen, and `R` restarts everything including the timer.
- Display: the countdown shows whole seconds rounded up, so “Time: 1” is the last second and the attempt fails at 0.

One thing to know: each frame's `dt` is capped at 0.1 s. If the window freezes (for example while dragging it), the timer doesn't jump to zero and kill you. The cost is that the clock pauses slightly during very long stalls.

give me the entire code

Only `game/game_engine.py` changes for Task 4. Replace the whole file with this:

python

Python

```python
"""
GameEngine: owns the frog and all vehicles, and runs one
frame's worth
of game logic.

Features: a 3-life system with a hit effect and respawn, a
Game Over
state, goal/score tracking with a win state, and a 30-second
countdown
per attempt. Collision detection lives in game/collisions.py.
"""

import math
import random
import time

import pygame

from game.frog import Frog
from game.vehicle import Vehicle
from game.collisions import check_collision
from game.renderer import (
    GRID_COLS, GRID_ROWS, GOAL_ROW, ROAD_ROWS, START_ROW,
CELL_SIZE, WIDTH, HEIGHT,
)

LANE_SPEEDS = [1.5, -2, 2, -2.5, 1.5, -2]   # one entry per
road row, alternating direction
```

```
STARTING_LIVES = 3
HIT_DURATION_FRAMES = 45   # ~0.75 s at 60 FPS; the game logic
is frame-based like vehicle speeds
SCORE_PER_CROSSING = 100
TIME_LIMIT = 30.0          # seconds per attempt
MAX_DT = 0.1               # cap on a single frame's elapsed
time (guards against stalls)


class GameEngine:


   def __init__(self):
        self._build_entities()

    def _build_entities(self):
        self.lives = STARTING_LIVES
        self.hit_timer = 0          # frames left in the hit
effect (0 = not being hit)
        self.game_over = False
        self.score = 0
        self.won = False
        self.time_left = TIME_LIMIT
        self._last_tick = time.perf_counter()
        self.hit_reason = None      # "vehicle" or "time", for
the on-screen indication

        start_col = GRID_COLS // 2
        self.frog = Frog(
            col=start_col, row=START_ROW,
            start_col=start_col, start_row=START_ROW,
            cols=GRID_COLS, start_row_limit=START_ROW,
        )
        frog_x_range = (start_col * CELL_SIZE, start_col *
CELL_SIZE + CELL_SIZE)

        self.vehicles = []
```

```
        for i, row in enumerate(ROAD_ROWS):
            speed = LANE_SPEEDS[i % len(LANE_SPEEDS)]
            vehicle_width = 40 if i % 2 == 0 else 70   # mix
of cars and wider trucks
            spacing = 300
            count = 2

            # Try a few random phases and keep the first one
that doesn't
            # already overlap the frog's starting column -
guarantees a
            # safe first lane instead of leaving it to chance.
            for _attempt in range(20):


                phase = random.randint(0, spacing - 1)
                 positions = []
                 safe = True
                 for n in range(count):
                     offset = phase + n * spacing
                     x = offset if speed > 0 else WIDTH -
offset - vehicle_width
                     positions.append(x)
                     if not (x + vehicle_width <=
frog_x_range[0] or x >= frog_x_range[1]):
                         safe = False
                 if safe:
                     break

            for x in positions:
                self.vehicles.append(Vehicle(x=x, row=row,
width=vehicle_width,
                                             height=CELL_SIZE
- 8, speed=speed))

    def handle_keydown(self, key):
        # R always works: restarts the whole game with full
lives, score 0 and a fresh timer.
```

```
        if key == pygame.K_r:
            self._build_entities()
            return

        # No movement during the hit effect, after game over,
or after winning.
        if self.game_over or self.won or self.hit_timer > 0:
            return

        if key == pygame.K_UP:
            self.frog.move(0, -1)
        elif key == pygame.K_DOWN:
            self.frog.move(0, 1)
        elif key == pygame.K_LEFT:
            self.frog.move(-1, 0)


       elif key == pygame.K_RIGHT:
            self.frog.move(1, 0)

    def update(self):
        # Real elapsed time since the previous update (not
frame counting).
        now = time.perf_counter()
        dt = min(now - self._last_tick, MAX_DT)
        self._last_tick = now

        # Vehicles keep moving the whole time, even during the
hit effect.
        for v in self.vehicles:
            v.update(road_width_px=WIDTH)

        # After a win or game over: no collisions, no life
changes, timer frozen.
        if self.game_over or self.won:
            return
```

```
        # Hit effect running: timer frozen, no collision
checks, respawn when it ends.
        if self.hit_timer > 0:
            self.hit_timer -= 1
            if self.hit_timer == 0:
                self.frog.reset()
                self.hit_reason = None
                self._last_tick = time.perf_counter()   # new
attempt starts now
            return

        if check_collision(self.frog, self.vehicles):
            self._lose_life("vehicle")
            return

        # Goal reached: register the crossing, add score, end
the game.
        # Checked before the timer so reaching the goal always
beats the clock.


       if self.frog.row == GOAL_ROW:
            self.score += SCORE_PER_CROSSING
            self.won = True
            return

        # Countdown for the current attempt.
        self.time_left -= dt
        if self.time_left <= 0:
            self.time_left = 0.0
            self._lose_life("time")

    def _lose_life(self, reason):
        self.lives -= 1
        self.hit_reason = reason
        if self.lives <= 0:
            self.game_over = True
        else:
```

```
            self.hit_timer = HIT_DURATION_FRAMES
            self.time_left = TIME_LIMIT      # each attempt
gets its own 30 seconds

    def draw(self, surface, font):
        from game import renderer
        renderer.draw_scene(surface, self.frog, self.vehicles)

        # Hit effect: frog turns red where it was hit (stays
red on game over).
        if self.hit_timer > 0 or self.game_over:
            pygame.draw.rect(surface, (220, 40, 40),
                             self.frog.get_rect(CELL_SIZE),
border_radius=6)

        renderer.draw_text(surface, font, f"Lives:
{self.lives}", (10, 10))
        renderer.draw_text(surface, font, f"Time:
{math.ceil(self.time_left)}", (WIDTH // 2 - 40, 10))
        renderer.draw_text(surface, font, f"Score:
{self.score}", (WIDTH - 140, 10))


       renderer.draw_text(surface, font, "Arrow keys to move.
R to restart.", (10, HEIGHT - 24))

        # Banners are drawn last so nothing covers them.
        if self.game_over:
            renderer.draw_banner(surface, "Game Over")
        elif self.won:
            renderer.draw_banner(surface, "You Won!")
        elif self.hit_timer > 0 and self.hit_reason == "time":
            renderer.draw_banner(surface, "Time's Up!",
size=48)
```

`renderer.py` stays as the version with the new `draw_banner(surface, text, size=72, color=...)`. `collisions.py`, `frog.py` and `vehicle.py` are untouched.

Claude is AI and can make mistakes.